

ECE 5760 Lab 2: Mandelbrot Set Visualizer

Md Shad, Michael Grillo, Stephen Barnett

March 6, 2026

1 Introduction

In this lab, we designed a Mandelbrot set visualizer on the DE1-SoC FPGA where we computed and rendered a high-resolution fractal in real time. The Mandelbrot set is generated by repeatedly calculating the complex iteration $z_{n+1} = z_n^2 + c$ using an initial condition of $z_0 = 0$, where each complex constant c corresponds to a pixel on the screen. For every pixel in the 640×480 display, the system determines whether the sequence diverges by evaluating whether the magnitude of z exceeds a threshold value. The color of each pixel is determined by the number of iterations required for the sequence to diverge. This produces the familiar fractal boundary and allows regions of stability and instability in the complex plane to be visualized.

The core computation was implemented in synthesizable Verilog and executed entirely within FPGA hardware using fixed-point arithmetic to efficiently perform the repeated multiplications and additions required for each Mandelbrot iteration. In order to accelerate the rendering, we designed a parallel architecture containing multiple iterator pipelines that operated simultaneously. Each of these iterators computed the iteration count for a smaller subset of the screen's pixels and stored the resulting color values inside dedicated on-chip M10K memory blocks. This organization of memory reduced contention between solvers and allowed the system to compute thousands of pixels concurrently.

The Hard Processor System (HPS) provided high-level control and enabled user interaction by communicating with the FPGA logic through memory-mapped parallel I/O (PIO) interfaces configured in Platform Designer. Through these registers, the HPS specifies the complex plane region that should be rendered, including the starting and ending real and imaginary coordinates as well as the step sizes between adjacent pixels. These parameters define the viewport of the Mandelbrot set and allow the user to reposition or zoom into specific regions of the image. Whenever the HPS signals that the viewport has been adjusted, the FPGA recomputes the fractal for the new coordinate range. A primary objective of this lab was to utilize as many FPGA hardware resources as possible, particularly DSP blocks and on-chip memory, in order to maximize parallelism and achieve the fastest possible rendering acceleration.

The computed pixel values are written into iterator-specific memory blocks, and the VGA controller reads and multiplexes the outputs of these memories to drive the 640×480 display. The system therefore renders the Mandelbrot set directly using FPGA-generated data without routing pixel values through the HPS. The display also supports interactive

exploration of the fractal, allowing users to pan and zoom into detailed regions of the set while observing the complex boundary structures that emerge. Users interact with the system through a mouse interface, which allows them to move a cursor across the display and select regions of the image to zoom into or pan across the complex plane. Through careful hardware parallelization, fixed-point design, and hardware–software co-design, our implementation demonstrated fast fractal rendering, efficient use of FPGA resources, and responsive user interaction through the HPS.

2 Mathematical Background

The Mandelbrot is defined as the parameter space where the equation through the complex space where $z_{n+1} = z_n^2 + c$ is bounded given an initial condition, $z_0 = 0$. For each value of $c \in C$, the complex sequence is iterated for a set number of iterations (usually 1000 iterations), and checked for divergence.

Typically, the Mandelbrot set is calculated over a small region $\in C$, where the fractal appears. This region is more precisely, values $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$. To represent these points in the equation $z_{n+1} = z_n^2 + c$, c is defined as $c = c_r + ic_i$, where c_r is the real component of c and c_i is the imaginary component of c .

The Mandelbrot solver iterates for each value within the rectangular region of the complex plane with $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$ and for each value, runs the iterative function until either the system "doesn't diverge" meaning that the system incurs 1000 iterations without exceeding the predefined bounds of divergence, which is mathematically determined to occur if $|z| > 2$.

Mathematically, the divergence is determined by evaluating the magnitude of the complex value $z = z_r + iz_i$, which is defined as $|z| = \sqrt{(z_r)^2 + (z_i)^2}$, or in addition, $|z|^2 = (z_r)^2 + (z_i)^2$. It has been previously proven that if $|z| > 2$, the sequence will diverge to infinity. Therefore, our solver checks if $|z|^2 > 4$ or $4 < (z_r)^2 + (z_i)^2$, and if so, this is the iterated where we know that the sequence has diverged.

To solve for the "Mandelbrot set visualization", each value of this rectangular region of the complex plane is calculated, and the output of each iterator, which is the iteration value where the outcome of the system is determined, whether that is the iterator value where we determine the system diverges, or 1000 if the system does not diverge is returned. To get the colorful images of the Mandelbrot that is commonly viewed online, each of these iterator values is mapped logarithmically to a specific color, with the value representing 1000, or convergence of the set, as the color black, and different colors representing difference speeds of divergence.

3 Roles of the ARM (HPS) and FPGA

A hardware–software co-design approach was used to implement the Mandelbrot visualization system, with the ARM-based Hard Processor System (HPS) handling system control and user interaction while the FPGA handled the computationally demanding fractal calculations. The HPS managed configuration parameters, user inputs, and communication with

the hardware accelerator, while the FPGA executed the highly parallel Mandelbrot iteration pipelines. This separation allowed specialized hardware to handle the computational workload while maintaining a flexible and interactive software interface.

3.1 FPGA Responsibilities

The FPGA implemented the core Mandelbrot computation engine. Multiple iterator pipelines were instantiated in hardware in order to evaluate many pixels simultaneously. Each iterator repeatedly computed the Mandelbrot iteration equation using fixed-point arithmetic, which allowed the arithmetic operations to efficiently map onto the Cyclone V DSP blocks and logic resources. These pipelines performed the multiplications, additions, and comparisons required to determine whether each complex value diverged and how many iterations were required before divergence occurred.

By executing these computations entirely in hardware, the FPGA provided deterministic execution and high throughput. The parallel iterator pipelines enabled the system to evaluate many pixels concurrently, making the rendering of the Mandelbrot set significantly faster than a sequential software implementation. Each iterator wrote the computed pixel color values into on-chip memory blocks, which were later read by the VGA subsystem to display the fractal image.

The FPGA also exposed a set of memory-mapped control registers that served as the interface between the HPS and the hardware accelerator. These registers stored parameters that defined the region of the complex plane being rendered, including the starting and ending real and imaginary coordinates, as well as the step sizes used between adjacent pixels. The FPGA continuously read these registers and used their values to determine the coordinates that each iterator pipeline should evaluate.

3.2 ARM (HPS) Responsibilities

The ARM processor running on the HPS acted as the high-level controller of the system. Rather than performing the Mandelbrot calculations directly, the processor was responsible for configuring the FPGA accelerator and managing user interaction. Through memory-mapped I/O, the HPS wrote the parameters that defined the viewing window of the Mandelbrot set, including the start and end coordinates and the step sizes for the real and imaginary axes.

The HPS also managed the interactive user interface. A C program running on the processor monitored keyboard input and adjusted the fractal viewing region accordingly. By modifying the coordinate ranges, users could pan across the complex plane or zoom into specific regions of the Mandelbrot set. Once these parameters were updated, the HPS wrote the new values to the FPGA registers, causing the hardware accelerator to recompute the fractal for the updated region.

Another responsibility of the HPS was monitoring system performance. The FPGA recorded the number of clock cycles required to compute a frame and stored this value in a hardware register. The HPS read this value, converted it into milliseconds using the FPGA clock frequency, and displayed the rendering time to the user. This allowed users to observe how quickly the hardware accelerator rendered the Mandelbrot set.

3.3 System-Level Integration

Separating computation and control between the FPGA and the HPS resulted in an efficient system architecture. The FPGA operated as a high-speed numerical engine dedicated to Mandelbrot calculations, while the HPS provided flexibility for configuration, interaction, and monitoring. Communication between the two components occurred through memory-mapped registers, which simplified debugging and system integration.

4 RTL Design

This design implements a hardware accelerator for generating a Mandelbrot set visualizer, using 4.23 signed fixed point arithmetic on an Altera DE1-SoC. This system primarily calculates the iterative Mandelbrot generation equation: $z_{n+1} = z_n^2 + c$, for values $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$, where $c = c_r + ic_i$. The components of z can be expanded into $z_{r,n+1} = c_r + (z_{r,n}^2 - z_{i,n}^2)$ and $z_{i,n+1} = c_i + 2z_{r,n}z_{i,n}$. Each clock cycle will correspond to a new iteration of the Mandelbrot equation. Our input values of c_r and c_i and the values of z_r and z_i are stored as bit size 27, with 4.23 signed fixed point arithmetic, an appropriate range as the Mandelbrot set iterations typically bounded with a range of ± 2 before escape, so the squared magnitude check occurs around 4. With 4.23 fixed-point, we allow for a sufficient integer range to protect against overflows, but additionally, allow for maximum resolution for zooming, accounting the DSP block's ability to do 27x27 bit multiply. Escape detection is implemented through checking whether the square magnitude, which is $|z|^2 = z_r^2 + z_i^2$, is $|z|^2 > 4$, which as mentioned in Section 2, has been mathematically proven as a point that diverges. A small issue regarding overflow errors occurred initially when testing in simulation, which led to an additional check before updating `zr_new` and `zi_new`, to ensure both values of $|z_r|$ and $|z_i|$ are < 2 , which could create overflow if not met, to ensure that proper treatment occurs when escape detection takes place. When $|z|^2 > 4$, our iteration is terminated and iteration count is recorded for the c value alongside a ready flag.

4.1 "Classic" 3 DSP Block Iterator

The baseline Mandelbrot iterator was implemented with a datapath that used 3 DSP blocks per cycle, which each preformed a 27x27 bit multiplication of the 4.23 fixed-point signed values. By evaluating the Mandelbrot set with two separate components, z_r and z_i , separate equation of $z_{r,n+1} = c_r + (z_{r,n}^2 - z_{i,n}^2)$ and $z_{i,n+1} = c_i + 2z_{r,n}z_{i,n}$ were performed in parallel.

These three multiplications were completed using an inferred DSP block using a reusable `signed_mult` module, generating a 54-bit intermediate product then resizing this product through bits [49:23], maintaining the original 4.23 bit signed fixed-point representation.

All values were represented in 4.23 fixed-point representation with 27 bits. This allowed for the use of only 3 27x27 bit multiplies needing to be completed each cycle, fully utilizing DSP blocks to maximize the potential speed and resolution of calculations. This design calculates one Mandelbrot iteration each clock cycle and the three DSP blocks are used to calculate the required products of $z_{r_{new}}^2$, $z_{i_{new}}^2$, and $z_r z_i$. $z_r z_i$ is used in the calculation of $z_{i_{new}}$, which is squared alongside $z_{r_{new}}$.

These squared results are written into registers at the clock edge and; therefore, are available at the next iterative cycle for calculations as z_r^2 and z_i^2 . These values can now be used for calculating new values of z_r and z_i without having to recalculate the squared values which were necessary to compute in order to check if escape conditions have been previously met.

Reset values in this iterator followed that z_r , z_i , z_r^2 and z_i^2 would all be reset to 0 alongside the "counter", "escaped", and "ready" signals. Parameters were set to `MAX_ITERATIONS` = 1000, demonstrating an assumption that if after 1000 iterations has occurred and the escape conditions have not been met, it can be assumed that the Mandelbrot iterator at that coordinate will likely converge. For each iteration that escape conditions or `MAX_ITERATIONS` are not met, values of z_r , z_i , z_r^2 and z_i^2 are assigned, a check for $|z|^2 > 4$ occurs, and values are recalculated.

This 3 DSP block architecture served as the baseline design for this project, as the architecture is simple yet efficient at iterating once per clock cycle and allowing for the possibility of 29 iterators being able to be instantiated and work together in parallel. However, as seen in the following subsections, with clever tactics, less DSP blocks per iterator can be generated at the cost of more clock cycles per iteration.

4.2 Generating Multiple Iterators

Our first attempt at generating the Mandelbrot set using our hardware used only one iterator. However, we were aware that running a single iterator was only a stepping stone, so we were careful to parameterize every aspect relating to the iterators on a local parameter - `NUM_ITERATORS`, which we will call **N**.

We transitioned to two iterators by moving our M10K block instance, iterator instance, and state machine into a generate block with for loop. We also declared arrays of each per-iterator signal so that we would be able index the IO of each iterator using only the corresponding genvar.

4.2.1 M10K Memory Modifications

Further, the M10K module itself was modified so that instances of the module only allocated enough memory to store the number pixels each iterator was responsible for. The equations used to find this number are described below.

$$MEMSIZE = \$ceil(\frac{WIDTH}{N}) * HEIGHT$$

Where `WIDTH` and `HEIGHT` are the screen size in pixels (640x480, respectively). We dispatch pixels to each iterator serially, in step sizes equal to **N** pixels. This means that each iterator handles at most 640 divided by **N** pixels in each of the 480 rows. By allocating this amount of memory for each iterator, we avoided using excess memory and kept the process of muxxing the color input to the VGA relatively simple, which will be discussed later.

4.2.2 Serial Writes and Multiplexed Reads

To keep track of the pixel data computed by each iterator, we developed a system for translating addresses of pixels requested by the VGA to a specific iterator instance i and a local memory address.

Each iterator writes to its unique memory block serially - incrementing the write address by one in most cases. Because some iterators will operate on one less pixel per row than others, we manage the 'x' and 'y' components of the write address independently. The 'x' component refers to the index of the pixel, out of the $\lceil \frac{W}{N} \rceil$ pixels per row that an iterator is responsible for. The 'y' component is similar, but keeps track of the row a pixel belongs to, storing it as a multiple of $\lceil \frac{W}{N} \rceil$. We increment these components every time a new pixel is dispatched to the iterator, keeping them synchronized with the X and Y coordinate of the pixel being computed. Every time this happens, the 'x' component is incremented by 1 (unless moving to the next row), and the 'y' component is incremented by $\lceil \frac{W}{N} \rceil$ (unless wrapping around to the first row). This system allows us to avoid an unnecessary multiply in our datapath (which was part of our critical path at one point) and construct the write address with a single addition.

When the VGA requests a pixel, we now have to decode it's X and Y coordinates to find the memory bank holding its color data and the local address. The memory bank is simple to retrieve, we do it by taking the screen width (640) modulo the current X coordinate. Our dispatch routine was designed specifically to be the reciprocal of the modulo operator, so this just works. The local address is rebuilt in one (VGA_pll) clock cycle using the general formula that we implemented sequentially on the iterator side. We multiply the row (Y coordinate) by the length of a row in each memory bank ($\lceil \frac{W}{N} \rceil$) and add that to the closest multiple of N that is less than the X coordinate ($\frac{X}{N}$). We implemented this equation explicitly, which Quartus was able to synthesize without using any additional DSP blocks. Since this calculation only needs to keep up with the VGA clock (much slower than our M10K PLL), we also were not concerned with the speed. After doing this computation, we're able broadcast the read address to all banks, and multiplex only the result of the desired bank into the VGA pixel data.

See Figures 1 and 2 for a visual representation of the banked memory architecture and dispatch.

		X								
		0	1	2	3	4	5	...	638	639
Y	0	1	2	3	4	1	2	...	3	4
	1	1	2	3	4	1	2	...	3	4
	2	1	2	3	4	1	2	...	3	4
	3	1	2	3	4	1	2	...	3	4
	4	1	2	3	4	1	2	...	3	4
	...	1	2	3	4	1	2	...	3	4
	480	1	2	3	4	1	2	...	3	4

Figure 1: Example diagram of pixel dispatch with 4 iterators on a 640x480 display. Cell contents indicate iterator instance identifier.

(X,Y)	0	1	2	...	159
0	(0,0)	(0,4)	(0,8)	...	(0,636)
1	(1,0)	(1,4)	(1,8)	...	(1,636)
2	(2,0)	(2,4)	(2,8)	...	(2,636)
3	(3,0)	(3,4)	(3,8)	...	(3,636)
...	...	...	...	...	...
480	(480,0)	(480,4)	(480,8)	...	(480,636)

Figure 2: Example of M10K bank for iterator 1 based on example dispatch in Figure 1. Cells represent color data for pixel at (x,y)

4.2.3 Problems

We hit several speed bumps when designing a scalable system for this lab. The first being memory constraints. We discuss our working solution in Section 4.2.1, however our first attempt was more naive. Our first approach was brute force - allocate enough memory for the entire pixel buffer for each iterator, then address pixels directly using their X and Y coordinates. This worked for one iterator, but it does not work for two. There are 307,200 pixels on the VGA screen - each taking up 1 byte in memory. Therefore, one buffer for the entire VGA screen takes up approximately 307KB, or 2,456Kb. Our FPGA only has 397 M10K blocks, making up 3,970 Kb of M10K memory total. If we tried to allocate the entire pixel buffer for each iterator we would quickly exhaust the available resources. As previously discussed, we solved this by allocating only the amount of memory required for each iterator and translating the addresses.

Another obstacle we encountered early on was properly maintaining the Mandelbrot input values (cr and ci) with more than one iterator. Our first attempt involved independently incrementing cr and ci for each iterator without tracking the corresponding X and Y. This meant handling all the incrementing and wrapping independently of the write address. This was a poorly thought out approach, and it presented several interesting bugs. The first visual we managed to display looked similar to TV static (see Figure 3), and it became clear we were losing track of our pixel coordinates somewhere. Our second attempt kept track of the X and Y coordinates plus a starting pixel, but allowed iterators to wrap around to the next row and carry over whatever excess cr that accumulated when incrementing past the end of the range. This generated a, somewhat interesting, skewed Mandelbrot set (see Figure 4). After this, we came up with the refined dispatch method discussed in Section 4.2.2 and generated a properly oriented Mandelbrot.

One other bug we encountered during this phase was several columns of pixels appearing to wrap from the right side of the VGA display to the left. The problem turned out to be a one-off issue with our write address translation, caused by incrementing the write address one cycle too early. We solved this by registering the correct write address on the cycle that the



Figure 5: Mandelbrot graphical bug with 87 iterators at 150MHz.

pixel is computed. Although this problem stuck around for several days, we unfortunately did not get a photo of it.

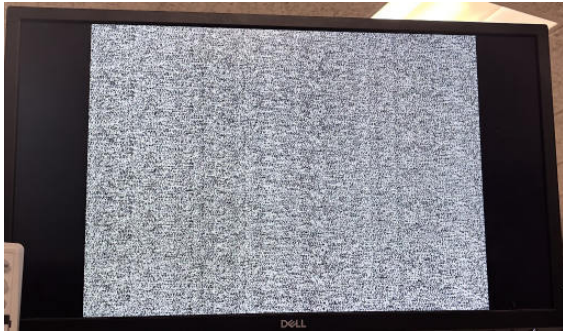


Figure 3: TV static created by first attempt at pixel dispatch.

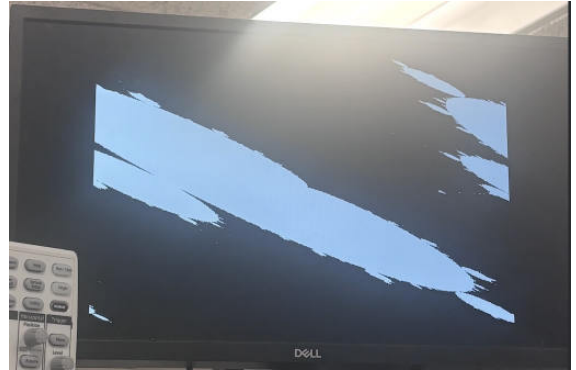


Figure 4: Skewed Mandelbrot set generated from a second, still undercooked, dispatch routine.

Graphical labs are very interesting because there is a high chance that any bugs in the software/hardware result in something that is visually interesting. Another bug that produced an intriguing visual result was excessive overclocking. After implementing the 3-cycle design discussed in Section 4.3.2, we were ambitious about the frequency capabilities of our new design. Naturally, we cranked the iterator/M10K clock up to 150 MHz and observed what you see in Figure 5

4.3 Attempted Reduced-DSP Iterator Optimization

We attempted to speed up generation of the Mandelbrot set by increasing parallelism. We designed two variations on the standard 3-DSP iterations, taking inspiration from multi-cycle FSM designs.

4.3.1 Two DSP - Two Cycle Iterator

Our first attempt at increasing parallelism uses only two DSP blocks, utilizing a hybrid-FSM design. One DSP handles calculation of $Z_R^{N+1^2}$ while the second computes $Z_R^N * Z_i^N$ in the first cycle and $Z_i^{N+1^2}$ in the second cycle. The motivation behind this design was to eliminate the critical path that passed through two DSP blocks. The goal was to increase the clock frequency by only doing one multiply per cycle and increase the total number of iterators running in parallel.

We achieved a modest increase in clock frequency as a result (10 MHz) but a net slow-down in Mandelbrot generation. With 43 iterators (using 86/87 DSP blocks on the DE1-SOC) we generated the full Mandelbrot set in 20 ms, approximately 6 ms slower than our original 3 DSP design.

This design has several obvious flaws in retrospect. Firstly, we were only able to achieve a $\sim 10\%$ increase in clock frequency and roughly a 50% increase in the number of iterators running. The tradeoff for these improvements was a 100% increase in the number of cycles required per iteration (from 1 to 2). Had we been able to achieve closer to 100% increase in clock frequency, this design may have had a chance at providing real improvement - however, the critical path in the modified design laid in a long chain of combinational logic spanning from *cr_in* on the Avalon Bus to the output register of the multiplexed multiplier. This path bottle-necked our frequency gains and indicated that a pipelined design may have been the better route for optimization. Secondly, because two is not a factor of 87, we were not able to utilize all 87 DSP blocks on the FPGA (with 43 iterators we used only 86). This obviously is sub-optimal as it means we weren't able to utilize a portion of the FPGAs compute.

4.3.2 One DSP - Three Cycle Iterator

Our second attempt at increasing parallelism stayed with the multi-cycle philosophy, but attempted to fix the flaws in the two-cycle design by fully committing to the FSM design rather than a hybrid design. Our main goal was to 3X the number of iterators we could run relative to the base design and keep the moderate increase in clock frequency. We wanted to maintain the same ratio of iterators to cycles-per-iteration as the baseline design, something that we failed to do in the two-cycle design. In theory, a design with 87 3-cycle iterators will compute the Mandelbrot set at the same speed as a design with 29 1-cycle iterators - all other factors held constant. Therefore, if we could sustain an increased clock frequency, we would see some gains.

While this design did yield some improvement over the two-cycle design, it still failed to surpass the base design. With 87 iterators, we could generate the whole Mandelbrot set in ~ 18 ms. A postmortem on this design also reveals several flaws in the multi-cycle principle.

Firstly, instantiating 87 iterators ballooned the area and routing requirement of the total design. On the first compile of this design, the Quartus fitter failed routing after roughly an hour of repeated attempts with "increased optimization." To get around this, we had to slightly tweak the Quartus compiler - telling it to prioritize constraining area rather than performance (speed of compilation). This tweak allowed the Fitter to succeed with 87 iterators and 81% logic utilization on the FPGA. While this is not inherently a flaw in this

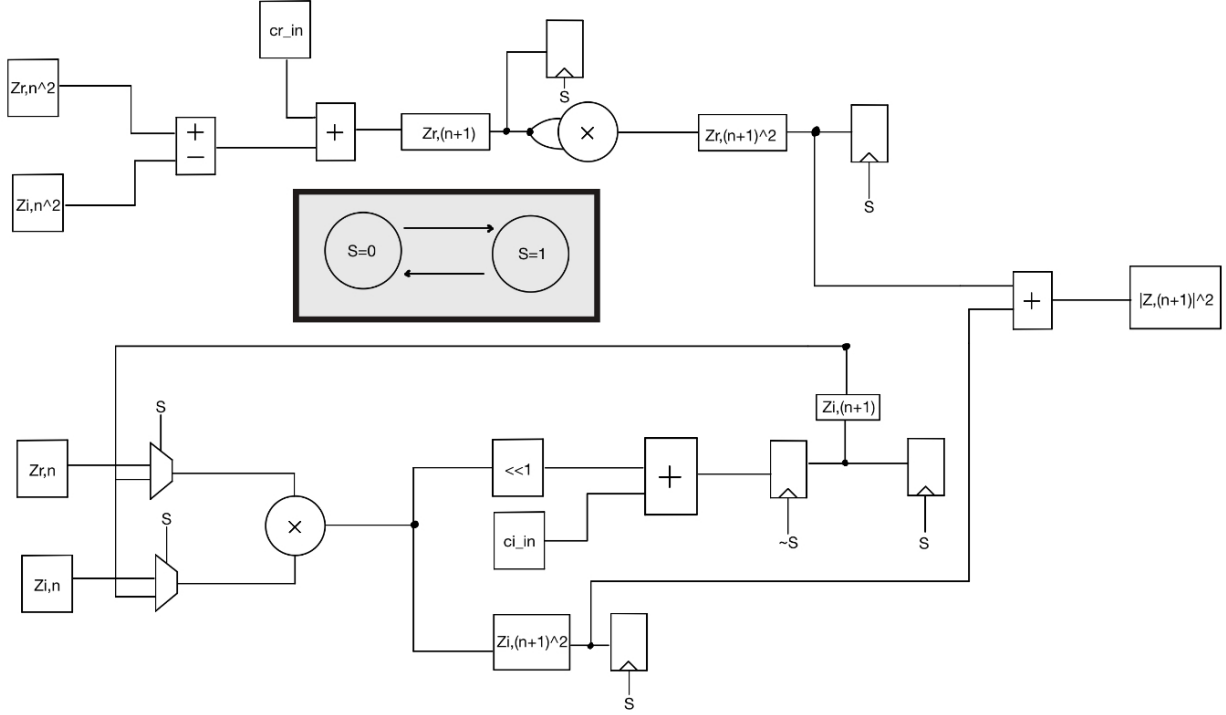


Figure 6: Block Diagram of two-cycle hybrid FSM Design. Including State Diagram and control signals.

design, it makes it clear that this design likely has much longer critical paths than previous designs - largely due to congested routing on the FPGA. Unfortunately, this turned out to be true and we were unable to meet timing with the 130 MHz clock that worked on our two cycle design. We reduced the clock to 120 MHz (the frequency of the base design) and achieved the 18 ms figure discussed previously.

This still leaves approximately 4 ms of time loss unaccounted for - if our working theory was correct. We find it likely that tweaks to the Quartus compiler and/or flaws in the load balacing of our design account for this discrepancy. While still unclear exactly what caused this time loss, this experiment combined with our discussions with other groups reinforces our conclusion that pipelining would have been the more optimal design philosophy for this problem.

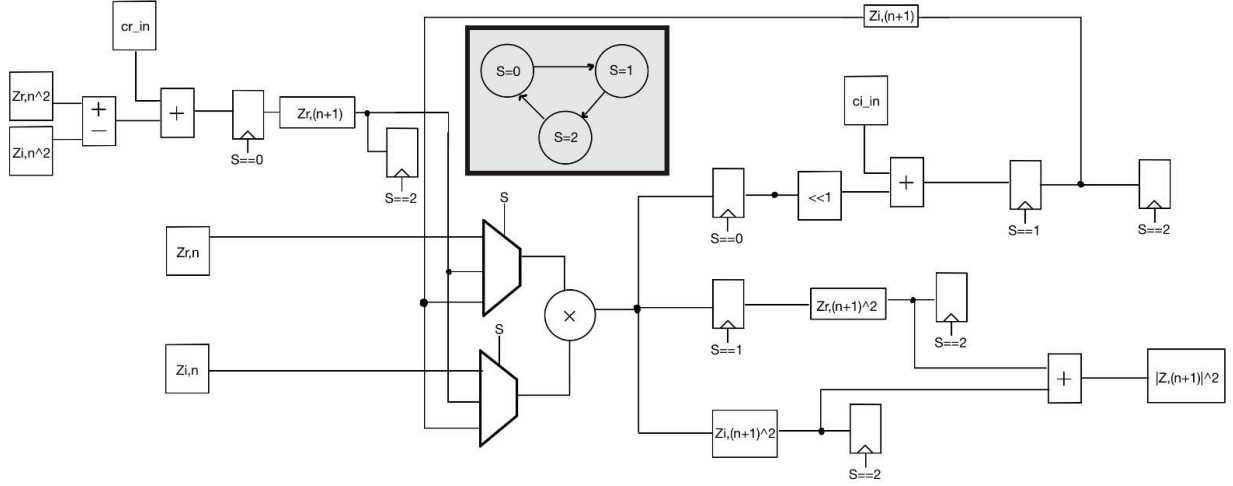


Figure 7: Block Diagram of three-cycle, single DSP iterator design. Including State Diagram and control signals.

5 Design and Testing Methods

5.1 Hardware Testing

The primary form of hardware testing used to test the Mandelbrot Verilog design was through comparison with a Python script which calculated the number of iterations before divergence of the Mandelbrot set iterator. Both the hardware and software implementations, the system iteratively calculates $z_{n+1} = z_n^2 + c$, with constant initial condition $z_0 = 0$. While the Python script used floating point arithmetic compared to the 4.23 fixed-point arithmetic in the hardware implementation, given matching values for c_r and c_i , the number of iterations before divergence remained equal between the Python implementation and hardware testing. Additionally, to check the values of the intermediate calculations in Python vs Verilog, the floating point values in python were scaled by 2^{23} so that the floating point values matched the fixed point values and comparison of intermediate values of z_{n+1} can be compared.

To test the system, a non-synthesizable Verilog testbench was created, defining as our device under test (DUT) as our Mandelbrot iterator, and initializing a clock and a reset signal before inputting scaled values of c_r and c_i to match 4.23 fixed-point format. Once reset was enabled and input values were defined, the system began iteratively computing new values of z_r and z_i each clock cycle, while determining if the escape condition has been met, so that the iteration counter can be set as ready.

An example waveform sourced from additional testing on Icarus Verilog waveform viewer is shown below, where these input values of c_r and c_i are shown alongside the counter value incrementing until ready is asserted and intermediate calculations occurring. Through comparison of both the counter values and the intermediate values with the Python scripts detailed below allowed for confident verification of our Mandelbrot iterators.

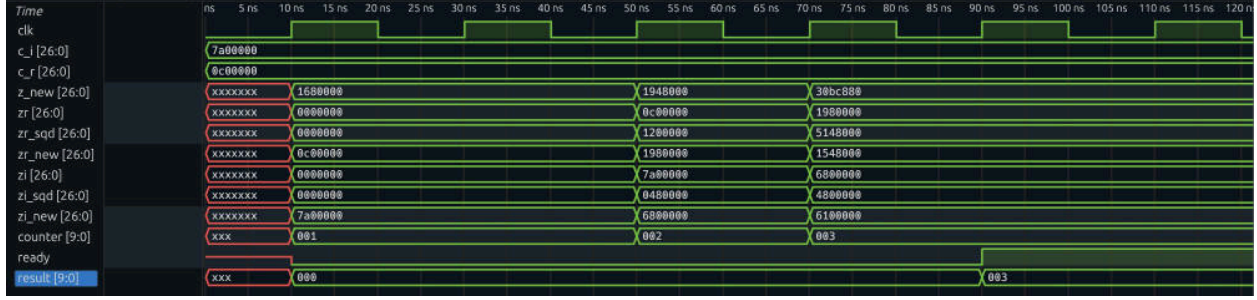


Figure 8: Waveform of Single Iterator

Additionally, to ensure consistency between the software implementation of the Mandelbrot solver and the dedicated hardware module, a variety of coordinates were tested including those that were determined to diverge rapidly, converge, and gray areas that may or may not converge, ensuring that edge cases of c_r and c_i values are tested properly. An example of such case could be when $c_r = -1.99$, and $c_i = -1.49$, the bottom left corner of the Mandelbrot set, which is expected to diverge incredibly rapidly.

Another example of a convergent case occurs when $c_r = -0.65$ and $c_i = 0.88$. Such a case's iteration projection in the complex plane is shown in the below figure, of which's code was sourced from [1]. In addition to the provided viewer, which allowed us to visually spot interesting edge cases and cases of extreme convergence and divergence, a separate Python script was used, which provided the values for $|z| * 2^{23}$, allowing for cross referencing with ModelSim.

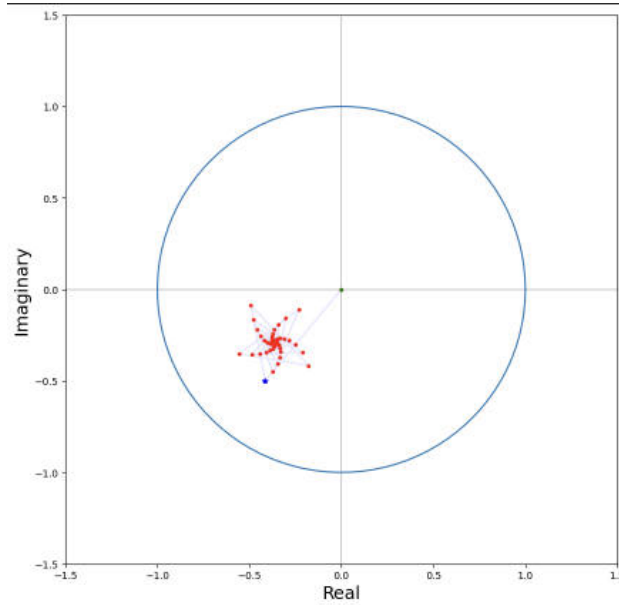


Figure 9: An Interesting Case of Convergence for the Mandelbrot Iterator

Finally, to confirm the correctness of the number of iterations before convergence, a third Python script was run for each selected value, providing the maximum number of iterations

before divergence of the Mandelbrot set, for true cross referencing of the iteration counter value of the Verilog module when the ready signal was enabled.

Outside of the combined system, small modules such as our signed multipliers were tested prior to ensure proper concatenation occurred of the 54 bit long multiply intermediate. Additionally, the waveforms of each signal were verified by comparing the output values with the expected values for each value tested.

Overall, the Mandelbrot hardware design was validated through comparison with a Python reference model, waveform inspection in ModelSim, and visual verification of correct behavior for signals such as ready and reset. A variety of input conditions were tested and both intermediate values and expected return values were both verified, allowing for confidence in the system generation at scale.

5.2 Software Testing

The software testing for this lab involved verifying that our mouse control was functioning properly, the HPS was writing to the correct memory addresses on the Avalon bus, and writing the correct values in fixed point form. We our mouse control using print statements which we could view over SSH. We verified that when the mouse was moved we received the expected feedback from our C program. In doing this, it became clear that we were properly interfacing with the mouse.

Verifying that we were writing the proper values to the proper memory addresses involved mostly visual inspection, and some print statements to reveal the internals. Every time our C loop runs, it reads back the values written to the FPGA - `cr_start`, `ci_start`, `cr_stop`, and `ci_stop` - and prints them to the console. We convert these values from fixed point to floating point and format them as ordered pairs representing the four corners of our viewport. This allowed us to verify that when our system resets, it does indeed show the Mandelbrot set in the range (-2.0, -1.0) to (1.0, 1.0). We also verified that when we zoomed and panned, our ranges changed as expected. Additionally, we had to verify that when we zoomed or panned from the HPS, we were telling our hardware to restart the render timer. We also read the timer output from our hardware and printed it to the console. Every time we clicked the mouse to zoom/pan we observed the timer change to represent how quickly the solver was able to render that portion of the set.

To verify that the signal being sent from the FPGA to the HPS was valid (the HPS reset signal) we verified that when KEY0 was pressed, our C program reacted accordingly and reset the viewport to the default range.

6 Results

In this lab, we achieved a best time of 14 ms to compute the Mandelbrot set in the default range. We did this with 29 iterator instances, utilizing all 87 DSP blocks on the DE1 FPGA.

Additionally, we designed a system that is scalable up to a theoretical 639 iterators (after which some minor tweaks would be required) given an FPGA with sufficient hardware resources. We managed to compile and run a design with 87 3-cycle iterators working

concurrently to generate the Mandelbrot set - exhausting the routing resources of the DE1 FPGA.

During this lab we learned about hardware limitations and the trade offs of multi-cycle architectures. The results of this lab are further discussed in the following sections, notably sections 7 and 8.

7 Chip Planner Layout

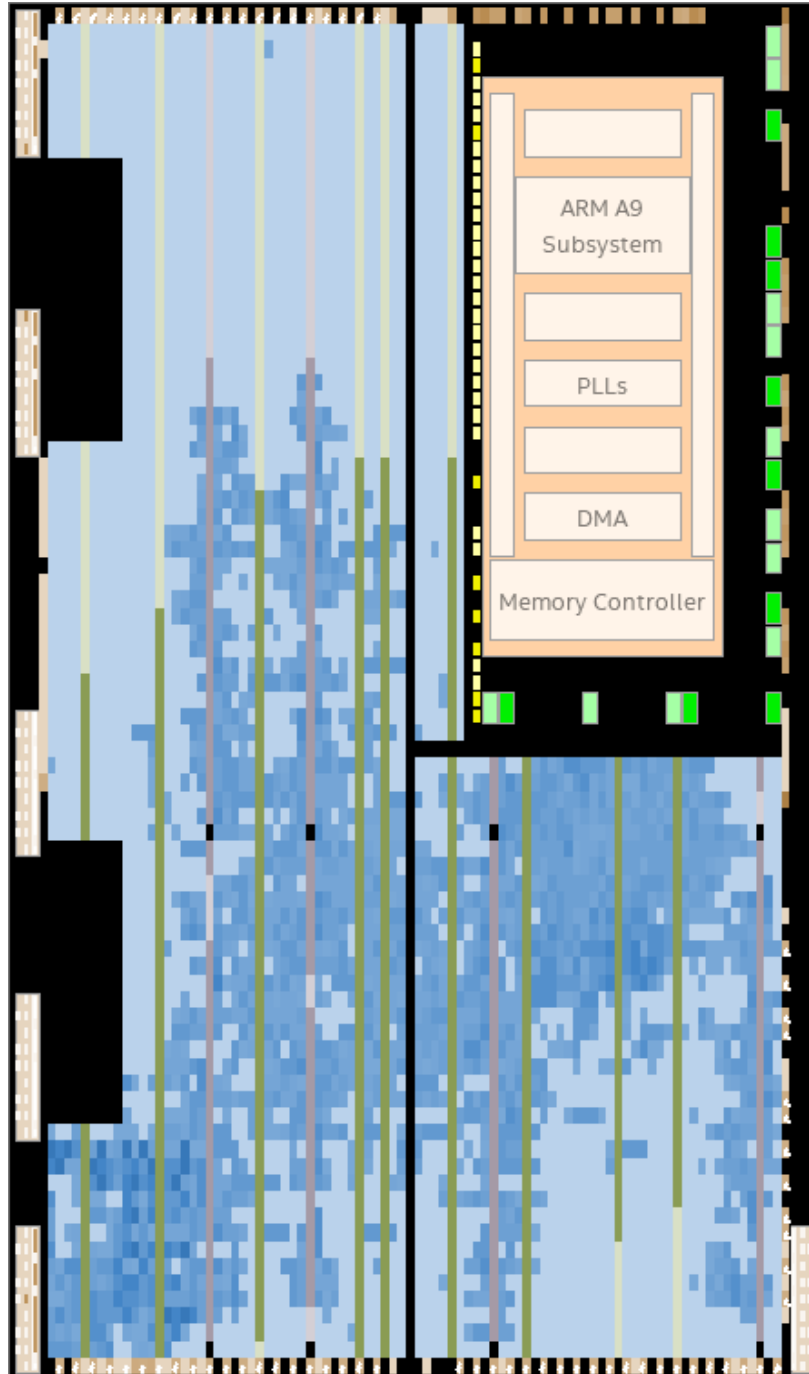


Figure 10: FPGA Chip Planner showing resource utilization of the Mandelbrot accelerator.

Figure 8 shows the Chip Planner view generated by Quartus, which provides a physical representation of how the Mandelbrot accelerator is mapped onto the FPGA fabric of the DE1-SoC. Unlike the RTL schematic, which primarily demonstrates the logical connectivity of the system, the Chip Planner shows the physical placement of logic elements, embedded

memory blocks, and routing resources after synthesis, placement, and routing have been completed in Quartus.

The large rectangular region on the right side of the planner corresponds to the Hard Processor System (HPS), which contains the ARM A9 subsystem, PLLs, DMA controller, and memory controller. This region is fixed silicon on the SoC, meaning it is not part of the programmable FPGA fabric. The remaining portions of the figure represent the programmable logic area where our Mandelbrot accelerator hardware was implemented.

The blue regions across the FPGA indicate areas where logic resources such as adaptive logic modules (ALMs) and registers are utilized by the design. In our Mandelbrot system, these resources correspond to the iterator pipelines, pixel dispatch logic, arithmetic datapaths, and VGA interface modules. The green vertical columns represent embedded memory resources such as M10K blocks, which are used in our design to store the pixel color values computed by the iterator pipelines before they are read by the VGA controller for display.

From the layout, we can observe that a large portion of the FPGA fabric is utilized, and the synthesis and placement tools distributed the logic resources across a wide region of the chip rather than concentrating them in a single area. This distribution helps reduce routing congestion and improves timing performance by keeping related logic and memory blocks relatively close together. The routing paths visible throughout the device illustrate how signals move between the iterator pipelines, memory blocks, control logic, and the VGA subsystem.

Poor placement could lead to long routing paths, increased propagation delays, and potential timing violations. However, as shown in the Chip Planner in Figure 8, the Mandelbrot accelerator successfully maps to the FPGA hardware, and the logic and memory resources are efficiently distributed across the fabric. This layout supports the highly parallel computations required for the fast fractal rendering achieved in our design.

8 Hardware Resource Utilization

Flow Summary	
Flow Status	Successful - Thu Feb 26 16:19:10 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	DE1_SoC_Computer
Top-level Entity Name	DE1_SoC_Computer
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	11,138 / 32,070 (35 %)
Total registers	14544
Total pins	368 / 457 (81 %)
Total virtual pins	0
Total block memory bits	2,817,512 / 4,065,280 (69 %)
Total DSP Blocks	87 / 87 (100 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	3 / 6 (50 %)
Total DLLs	1 / 4 (25 %)

Figure 11: Hardware Utilization

Quartus Prime reports that our synthesized Mandelbrot accelerator uses 11,138 adaptive logic modules (ALMs) out of 32,070 available (35%), along with 14,544 registers. The design also uses 2,817,512 block memory bits out of 4,065,280 (69%), and 87 DSP blocks out of 87 available (100%). In addition, the system utilizes 368 I/O pins out of 457 (81%), 3 PLLs out of 6 available (50%), and 1 DLL out of 4 available (25%).

These hardware utilization statistics reflect the architecture implemented for our Mandelbrot visualization system, which relies on a large number of parallel iterator pipelines implemented in FPGA hardware. Each iterator continuously evaluates the Mandelbrot iteration equation using fixed-point arithmetic and performs several multiplications and additions every clock cycle. We intentionally structured the design to maximize parallelism, which required many hardware arithmetic units to be instantiated simultaneously.

The ALM usage primarily comes from the control logic, iterator pipeline logic, pixel dispatch circuitry, and VGA interface modules. Each iterator pipeline contains arithmetic datapaths, iteration counters, comparison logic for divergence testing, and control state machines that determine when a pixel has finished computing. In addition, the VGA subsystem requires logic resources to generate display timing signals, manage pixel coordinates, and multiplex the outputs from multiple iterator pipelines to the display memory.

The large number of registers used in this design is expected because the Mandelbrot computation and VGA display logic are fully synchronous. Registers are required to store intermediate arithmetic values within the iterator pipelines, maintain iteration counters, track pixel coordinates, and hold pipeline stage values used in the arithmetic datapath.

The DSP block usage is especially significant in this design, as all 87 available DSP blocks are utilized. These DSP blocks implement the fixed-point multiplications required for the Mandelbrot iteration, such as computing z_r^2 , z_i^2 , and $2z_r z_i$. Quartus maps these operations to dedicated hardware multipliers because they are faster and more efficient for multiplication than implementing these operations using general-purpose logic elements. The full utilization of DSP resources reflects the highly parallel nature of the design and the large number of iterator pipelines instantiated to maximize computational throughput.

The block memory utilization of 69% is primarily associated with storing the pixel color values produced by the iterator pipelines. Each iterator writes the computed color value of the pixels it processes into on-chip M10K memory blocks. These memory buffers allow the system to store intermediate pixel results while the VGA controller reads the data to display the Mandelbrot image on the screen.

Finally, the I/O usage is relatively high because the system interfaces with multiple external components on the DE1-SoC board, including VGA output, control inputs, and communication signals between the FPGA and the HPS. The PLLs are used to generate the required clock domains for the system logic and the VGA display timing used during rendering.

8.1 Critical Path

For our Mandelbrot accelerator system, we expect the longest combinational logic chain to occur within the arithmetic datapath of the iterator pipelines.

Each iterator repeatedly evaluates the Mandelbrot iteration equation

$$z_{n+1} = z_n^2 + c$$

using fixed-point arithmetic. During each iteration, the system computes the real and imaginary components of the next complex value using the expressions

$$z_{r,\text{new}} = z_r^2 - z_i^2 + c_r$$

$$z_{i,\text{new}} = 2z_r z_i + c_i$$

To perform these computations, the system requires several arithmetic operations within a single clock cycle, including multiple multiplications, additions, and subtractions. In particular, computing z_r^2 , z_i^2 , and $2z_r z_i$ requires fixed-point multiplications that are mapped by

Quartus onto dedicated DSP blocks. Although these multipliers are implemented in specialized hardware, the surrounding logic requires additional operations such as sign extension, truncation, shifting, and addition, which all contribute to the overall combinational delay.

Therefore, the critical path most likely occurs along the register-to-register path that begins at the current complex state registers storing z_r and z_i , passes through the arithmetic units responsible for squaring and multiplying these values, and ends at the registers that store the updated values for the next iteration.

A representative path in the datapath can be summarized as:

state register $\rightarrow$ multiplication $\rightarrow$ subtraction/addition $\rightarrow$ truncation/shift $\rightarrow$ next state register

Because multiple arithmetic operations may be chained together within the datapath, this path contributes to the longest propagation delay in the design. Therefore, the efficiency of the arithmetic pipeline plays a crucial role in determining the timing performance of the system. Understanding the critical path allows us to ensure that the design operates within the desired clock frequency range. If the chain of arithmetic operations becomes too long, the maximum achievable clock speed would decrease, which would limit the overall performance of the Mandelbrot accelerator.

By utilizing dedicated DSP multipliers and structuring the arithmetic datapath efficiently, our design is able to maintain a clock frequency that supports fast parallel computation across all iterator pipelines.

8.2 Program Listings

For this subsection of Verilog design, the 3 DSP block iterator will be primarily considered as a result of the 2 and 1 DSP block iterators failing to provide significant improvements in Mandelbrot generation times. The following table describes each of the primary modules used to generate the Mandelbrot Set Visualizer.

Table 1: Program Listing Table: Verilog Mandelbrot System Design

Module	Type	Description
<code>testbench_mandelbrot</code>	Testbench	Testbench module for simulation. This testbench generates clocks, applies reset sequencing, and initializes inputs the Mandelbrot iterator for Testing. This test bench instantiates <code>mandlebrot_system</code> as the DUT and allows for testing of the synthesizable <code>mandlebrot_system</code> module.
<code>signed_mult</code>	RTL (Combinational)	Signed fixed-point multiplier for the 4.23 format used for Mandelbrot iterations. Computes the full, 54 bit product, <code>mult_out = a*b</code> and truncates bits [49:23] to return the correctly scaled 27-bit fixed-point result.
<code>mandlebrot_system</code>	RTL (Datapath + State)	Implements a Mandelbrot iterator using the iteration $z_{n+1} = z_n^2 + c$. Computes the real and imaginary updates and the square of those updates, tracks the number of iterations, performs escape detection for the condition $ z ^2 > 4$, and outputs escape status and iteration result.
<code>M10K_1000_8</code>	Memory (On-chip RAM)	Per-iterator memory block used to store pixel information such as color data. Each iterator writes computed pixel values to its own M10K piece and this is later fed to VGA display.
<code>vga_driver</code>	RTL (Display Interface)	Generates VGA timing and synchronization signals to send to VGA monitor, reads pixel color data from M10K memory, and outputs RGB signals for VGA display.

The system generates a visualization of the Mandelbrot set by evaluating the escape time as described by number of iterations until the escape condition is met. For each coordinate on the screen, the `mandlebrot_system` iterates until either $|z|$ exceeds the escape threshold or `MAX_ITERATIONS` are reached. The resulting iteration count is then mapped to an 8 bit color value, stored in on-chip M10k memory and fed through VGA display driver to read these pixel values and display them on the VGA display. Points that do not diverge within

1000 iterations are colored black and as points escape, a gradient of colors are assigned, corresponding to the escape time, which produces the Mandelbrot's characteristic gradient patterns.

8.3 User Interface

The Hard Processor System (HPS) managed the user interface for the Mandelbrot visualizer using a C program that communicates with the FPGA through memory-mapped registers. The HPS software allows the user to interactively explore different regions of the Mandelbrot set by modifying the coordinates of the complex plane being rendered. These parameters are continuously written to control registers, which the FPGA Mandelbrot accelerator reads in order to recompute the fractal.

The C program accesses FPGA registers using Linux memory mapping through `/dev/mem`. By mapping the lightweight HPS-to-FPGA bridge into user space, the program obtains pointers to hardware registers that control the Mandelbrot rendering parameters. These registers contain the starting and ending coordinates of the real and imaginary components of the complex plane, as well as the step sizes used to move between adjacent pixels. During execution, the program continuously updates these values and writes them to the FPGA so that the hardware iterators can compute the Mandelbrot set for the selected region.

To allow interactive control, the program creates a separate thread that monitors keyboard input using the `ncurses` library. This thread allows the user to dynamically modify the viewing region of the Mandelbrot set. For example, when the user presses the `i` key, the current range of the complex plane is reduced to perform a zoom-in operation, while pressing the `o` key increases the current range to zoom out. Additional key commands allow the user to pan across the fractal: `w` and `s` shift the view vertically, while `a` and `d` move the view horizontally across the complex plane.

When the user adjusts these parameters, the program recalculates the corresponding step sizes for the real and imaginary axes and writes them to the FPGA control registers. The FPGA Mandelbrot accelerator then recomputes the fractal image using the updated coordinate range. Because the hardware iterators operate in parallel, the system can render the updated fractal view very quickly after each interaction.

The interface also provides performance feedback to the user. The FPGA records the number of clock cycles required to render the Mandelbrot set and stores this value in a hardware register. The HPS reads this value and converts it into milliseconds using the known clock frequency of the FPGA. This rendering time is displayed both in the terminal and on the VGA display, allowing users to observe the performance of the hardware accelerator while exploring the fractal.

Table 2: Program Listing Table: Mandelbrot Software Control Program

Module	Type	Description
<code>control_thread</code>	Software Thread	This is the keyboard control thread that we used for interactively navigating the Mandelbrot set. This thread uses the <code>ncurses</code> library to allow reading user key presses and adjusting the viewing window of the fractal by altering <code>cr_range</code> , <code>ci_range</code> , <code>cr_start</code> , and <code>ci_start</code> . This also supports zooming and panning across the complex plane while the FPGA continues to render.
<code>main</code>	Software Control / Host Interface	Top-level host program responsible for communication with the FPGA Mandelbrot solver. Opens <code>/dev/mem</code> , maps the lightweight HPS-to-FPGA bridge into the virtual memory, initializes pointers to the memory-mapped control registers, launches the thread for control, and writes updated Mandelbrot viewing coordinates to the memory so the FPGA can iterate. Additionally, it reads the FPGA timing counter and returns the rendering time for display on the VGA display.
Fixed-point conversion macros	Software Utility	Set of macros including <code>float2fix</code> , <code>fix2float</code> , <code>int2fix</code> , <code>fix2int</code> , <code>multfix</code> , and <code>divfix</code> . These provide conversion support for the 4.23 fixed-point representation used on the FPGA for Mandelbrot iterations. This allows for the software to correctly generate the scaled parameters necessary for proper interfacing with the FPGA through memory-mapped registers.

8.4 Documentation and Answers to Lab Questions

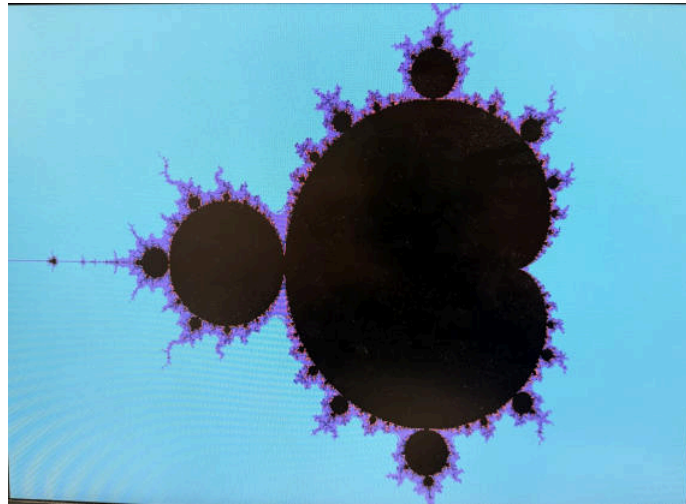


Figure 12: Rendered Mandelbrot set produced by our FPGA hardware accelerator.

8.5 A collection of photographs

For us, two areas on the Mandelbrot set stand out. These are the entrance to seahorse valley (Figure 13) and the mini Mandelbrot (Figure 14)

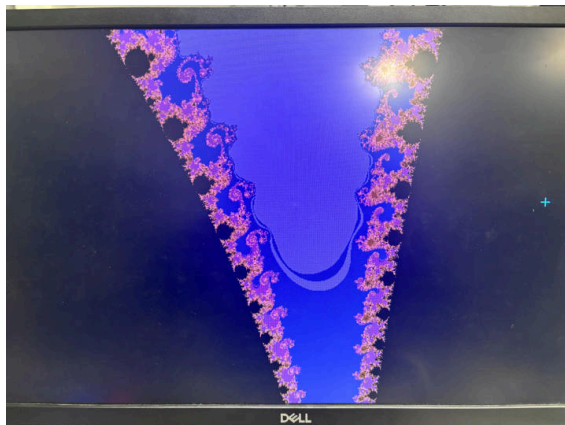


Figure 13: Entrance to "seahorse valley." A popular part of the Mandelbrot set. Range: $x=[-0.87, -0.68]$ $y=[-0.26, -0.13]$



Figure 14: Mini version of the Mandelbrot set. Lives to the right of the main set, and much smaller! Range: $x=[-1.48, -1.46]$ $y=[-0.009, 0.006]$

8.6 Timings for Various Regions

Table 3: Computation Times for Mandelbrot Set Regions

Requested Region	Actual x Range	Actual y Range	Time (ms)
$x \in [-2, 1], y \in [-1, 1]$	$[-2, 1]$	$[-1, 1]$	16.3
$x \in [-0.758, -0.75], y \in [0.05, 0.06]$	$[-0.759, -0.747]$	$[-0.047, -0.4]$	64
$x \in [-1.45, -1.3], y \in [-0.07, 0.07]$	$[-1.49, -1.3]$	$[-0.06, 0.06]$	25

8.6.1 Platform Designer

For this lab we configured several PIO ports on the AXI bus for communication between the HPS and the FPGA.

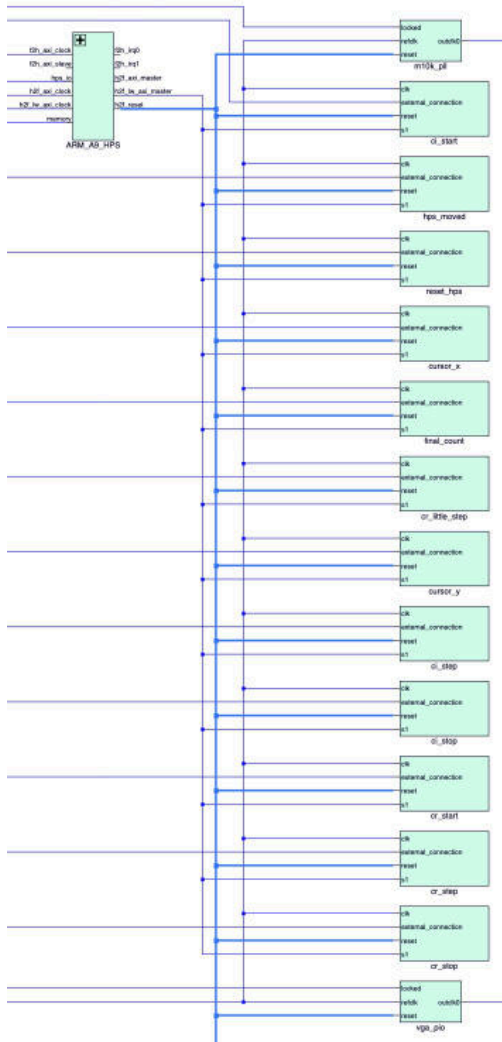


Figure 15: Quartus generated block diagram of the modules configured on the AXI Bus via Platform Designer.

- **cr_start:** Output, real component of the initial value for c .
- **ci_start:** Output, imaginary component of the initial value for c .
- **cr_stop:** Output, real component of the end value for c .
- **ci_stop:** Output, imaginary component of the end value for c .
- **cr_step:** Output, real component of step size between each pixel computed by an iterator.
- **ci_step:** Output, imaginary component of step size between each pixel computed by an iterator.
- **cr_little_step:** Output, real component of step size between each adjacent pixel in a row.
- **cursor_x,y:** Output, X and Y components of the mouse cursor on the VGA screen.
- **hps_moved:** Output signals FPGA to restart computation and timing when the viewport is moved.
- **hps_reset:** Input, signals HPS to reset all outputs to default states when FPGA is reset (KEY0).

Our QSYS design primarily facilitates communication from the HPS to the FPGA. Since all of the computation and VGA interaction was done from the hardware side, our C program was only responsible for sending over the initial conditions to generate and other UI necessities like the mouse cursor location. When the HPS detects a mouse click, it computes the new region for the viewport (a start point and a range), calculates an end point, and sends the viewport size to the FPGA. Then, the HPS tells the FPGA that it has requested a view change and tells it to reset using the `hps_moved` signal. As seen above, most of the PIO on the bus reflect this. The one exception is `hps_reset`, which tells the HPS when the FPGA has been reset (KEY0) so that it can reset to the default range. To ensure that this value is read properly from the C program, its declared as a volatile 8 bit integer to prevent the compiler from optimizing operations on this variable with caching.

9 Comments

This lab helped us understand the challenges involved in designing a hardware accelerator on an FPGA and how hardware parallelism can significantly improve performance for computationally intensive problems such as rendering the Mandelbrot set. Even though the mathematical formula for the Mandelbrot iteration may not seem difficult to implement, implementing it efficiently in hardware required careful consideration of the datapath design, memory organization, and parallel iterator pipelines. A large portion of our time was spent debugging, refining, and optimizing the system rather than simply implementing the basic iteration equation.

One of the most difficult challenges we encountered involved fixed-point arithmetic. While the Mandelbrot algorithm is straightforward when implemented in floating-point software, hardware implementations require fixed-point representations that must be carefully scaled. Small mistakes in scaling, bit shifting, or truncation often caused incorrect fractal patterns or unexpected divergence behavior. Debugging these issues required examining intermediate signals and verifying that the arithmetic operations were producing the expected results.

Another challenge was designing a parallel architecture that maximized the FPGA resources available while maintaining correct memory access patterns. Because multiple iterator pipelines compute pixels simultaneously, we had to carefully manage how pixel coordinates were distributed across the iterators and how the results were stored in memory. Ensuring that each iterator wrote to the correct memory location without conflicts required careful address calculations and memory organization. This process highlighted how important data flow and memory management are when designing high-performance hardware systems.

We believed that this lab was an amazing way to really get us thinking on how to optimize our designs where we reduced the number of logic parts needed in our design but to also exhaust the FPGA resources and take advantage of its capabilities.

10 Additional Questions

10.1 What determines how deep you can zoom?

The determining factor for the maximum depth primarily comes from the precision of our fixed-point numeric representation, but additionally, resolution is limited by our maximum amount of iterations and the VGA display resolution. Our design uses 4.23 fixed-point representation, where 23 bits are used for fractional precision. This means that the smallest possible step that can be represented is $\Delta \approx 2^{-23}$. Due to this, as we zoom in deeper than this precision allows, the nearby coordinates are numerically identical and the fractal can no longer be detailed. Additionally, as a fault of our fixed-point representation, at very high zoom levels, rounding errors from the fixed-point arithmetic accumulate leading to the possibility of divergent trajectories at very high levels of resolution. Additionally, our iteration limit of MAX_ITERATIONS, which in our iterator was set to 1000 could lead to resolution issues in terms of color, where we only had a few colors in a basic gradient that with more iterations and additionally more colors, could lead to higher resolution of our pixels in terms of color and better resolution in areas of interest where there is a region that has adjacent points with some diverging and some not-diverging in our interval. This will reveal more detailed boundaries. Lastly, our VGA display had resolution of 640x480, which allows for more visible detail while zooming but does not allow for additional numeric precision.

The depth to which the Mandelbrot set can be explored is primarily limited by the precision of the fixed-point representation, which in this design was 4.23 fixed-point which provides 2^{-23} resolution in the complex plane. To improve the possible depth that can be explored, the fixed-point representation can consist of more fractional bits; however, this would lead to a slower computation due to more computational resources being required.

11 AI Usage

We used artificial intelligence very limitedly, as we only used it to clarify our understanding, polish our work, and debug any mistakes or errors we were facing. For example, through the development process, AI was occasionally used to help interpret compiler and synthesis error messages and to assist in diagnosing issues that came up during debugging. Specifically, it was used to help identify syntax mistakes, resolve signal width mismatches and understand the potential causes for any incorrect logic in our hardware compilation and testing.

AI was also utilized as an aid when reviewing hardware and software implementation details. In particular, it helped clear up aspects of the Verilog design process, memory-mapped communication between the HPS and FPGA, and other system-level concepts when some unexpected behavior had occurred during development. When teaching assistants or professors were not present when we were working outside designated lab hours, AI served as a teaching assistant to clarify any information from the checkoff requirements that seemed confusing or required more context to understand. Moreover, we used AI to describe the functionality of several sample code that was provided to us such as the code for utilizing the mouse to zoom in and out of our design.

Furthermore, AI's role in writing this report was to polish our language by fixing any grammatical and spelling mistakes and modifying the flow of our sentences in our initial drafts written by us. All the information describing our implementation of this lab written in this report was developed with our own ideas and thoughts but was strengthened through the use of AI to allow clarity for the reader.

Prompt Interaction Log

Session Date: February 27, 2026

Topic: Understanding Linux mouse input code for HPS–FPGA design integration

Prompt #	Date	Prompt Summary	Edits	Insertions	Deletions	Mods Accepted
1	Feb 27, 2026	User requested an explanation of a C program that reads raw mouse input from <code>/dev/input/mice</code> .	0	0	0	0
2	Feb 27, 2026	User asked which sections of the code should be modified for their FPGA/HPS system design without providing code changes.	0	0	0	0
3	Feb 27, 2026	User requested generation of a prompt log including timestamps and counts of edits and code changes.	0	0	0	0

Table 4: Prompt interaction log for discussion regarding Linux mouse input and HPS–FPGA integration.

Session Summary

- Total Prompts: 3
- Total Edits Suggested: 0
- Total Code Insertions: 0
- Total Code Deletions: 0
- Total Code Modifications Accepted: 0

During this session, the interaction focused on understanding low-level Linux device input via `/dev/input/mice`, identifying which sections of the program would be modified for integration with an HPS-FPGA co-design, and generating a documentation-style prompt log. No direct code edits or modifications were requested or applied.

References

- [1] Van Hunter Adams, *ECE 5760: Mandelbrot Set Visualizer Lab*, Cornell University. Available: <https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot.html> Accessed: March 2026.